OPERATIONS HANDBOOK

VC

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

vSAN OSA Disk Group Rebuild Handbook

Verified per-host step-by-step procedure to retag misclassified capacity disks (SSD=False on flash media) and rebuild the disk group online via Maintenance Mode + Full Data Migration.

VSAN OSA | DISK GROUP | CAPACITYFLASH | MAINTENANCE MODE

vSAN 9.0 VMware vSAN OSA

vSAN OSA Disk Group Rebuild — Step-by-Step Handbook

Prepared by: Virtual Control LLC **Date:** May 8, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **Source Broadcom KB:** KB 315532 (manual disk group remove/recreate procedure)

Table of Contents

---

1. Executive Summary

This homelab's nested ESXi vSAN OSA cluster has 4 capacity disks reporting `Is SSD: false` despite running on physical NVMe/SSD media. vSAN treats those disk groups as hybrid (cache-then-HDD), causing 70 ms write latency and 0% Local Client Cache hit rate. This is the verified root cause behind the 6th VCFA deploy attempt failing (etcd slowdown → kube-apiserver PostStartHook timeout).

1.1 Definitive procedure — single path, verified-on-esxi02 2026-05-09

> **MAJOR REVISION 2026-05-09.** First execution on esxi02 proved that the SATP-rule + reboot path documented in earlier revisions of this handbook **does NOT work on nested ESXi** running on VMware Workstation. After running `esxcli storage nmp satp rule add ... --option "enable_local enable_ssd"` and rebooting, both target disks still reported `Is SSD: false`. The actual root cause is one layer up: the outer Workstation VM's `.vmx` file is missing `sataX:Y.virtualSSD = "1"` for the affected disks. The fix MUST be applied at the outer hypervisor's `.vmx` layer. Section 4 below documents the corrected procedure that completed successfully on esxi02 at 23:44 UTC 2026-05-09.

This handbook commits to **ONE procedure** per host:

1. Enter Maintenance Mode with **Full Data Migration** (vSAN evacuates the host's data; DRS vMotions running VMs)
2. Remove the existing disk group via `esxcli vsan storage remove -u <UUID>`
3. **Power off the nested ESXi VM gracefully** from VMware Workstation (host already in MM, vSAN already evacuated → zero cluster impact)
4. **Edit the outer `.vmx` file** — add `sataX:Y.virtualSSD = "1"` for each misclassified disk slot (the existing cache + first capacity disk already have this; the new capacity disks added later do not)
5. **Power on the nested ESXi VM** from VMware Workstation
6. SSH back in, verify `Is SSD: true` on each disk
7. Apply `esxcli vsan storage tag add -t capacityFlash` per misclassified disk
8. Re-create the disk group with `esxcli vsan storage add -s <cache> -d <capacity1> -d <capacity2> -d <capacity3>`
9. Verify BOTH `IsSSD=1` AND `IsCapacityFlash=1` per disk via `vdq -q`
10. (Optional) Verify from vCenter PowerCLI that capacity tier reports `SSD=True`
11. Exit Maintenance Mode
12. Wait for vSAN resync to fully zero before moving to next host

Per host wall time: 1-2 hours (no reboot needed — power-off/edit/power-on is faster). **Total for 3 hosts:** 3-6 hours.

1.2 Why the `.vmx` path (and not SATP rule + reboot)

The KB 341618 path (`esxcli storage nmp satp rule add ... --option "enable_local enable_ssd"` + reboot) is documented for *physical* ESXi hosts where the storage controller misreports a real SSD as rotational. SATP rules tag at the VMkernel layer, but for nested ESXi on VMware Workstation, the `Is SSD` flag is propagated FROM the outer hypervisor TO the guest via `sataX:Y.virtualSSD` in the outer `.vmx`. No amount of in-guest SATP manipulation can override the device-type metadata coming in from the outer Workstation VM. **Layer 1 (outer `.vmx`) wins.** This was proven empirically on esxi02 — SATP rule applied, host rebooted, `Is SSD` still `false`. After power-off → `.vmx` edit → power-on, `Is SSD: true` on first check.

The `capacityFlash` tag is still required after the `.vmx` fix — `IsSSD` and `IsCapacityFlash` are independent flags per Broadcom TechDocs (Mark Flash Devices as Capacity). The documented all-flash verification expects BOTH `IsSSD=1` AND `IsCapacityFlash=1` on each capacity disk.

Outcome after all 3 hosts complete: every capacity disk reports `IsSSD=1`, `IsCapacityFlash=1`, and vCenter PowerCLI shows `SSD=True`. vSAN runs as all-flash OSA. etcd write latency drops from ~3 sec to <100 ms. Future VCFA deploys complete normally.

Why this handbook previously had the wrong primary path. Earlier revisions cited Broadcom KB 341618 (SATP rule + reboot) as canonical because that is the documented path for *bare-metal* hosts. The handbook author did not initially recognize that nested ESXi on Workstation requires a Layer-1 fix instead. The 2026-05-09 esxi02 execution surfaced the discrepancy: SATP rule applied cleanly, host rebooted, but `Is SSD: false` persisted. Comparing the running `esxi01.lab.local.vmx` (SSD-correct) against `Clone of esxi02.lab.local.vmx` (SSD-broken) showed the outer-VM `virtualSSD` flags missing on the affected slots. This handbook is now corrected with the verified procedure as Section 4.

---

2. Confidence and Procedure Authority

Per the user's "no guessing" rule, every step in this handbook is sourced from a Broadcom KB or empirically verified on esxi02:

Step range	Source
Step 0 — resync wait	Internal memory `feedback_vsan_resync_first_check`
Step 1 — DRS	Standard vCenter DRS configuration
Step 2 — vCenter pre-migrate	Standard vCenter vMotion
Step 3 — Maintenance Mode + FDM	Broadcom KB 315532 + memory `feedback_vsan_mm_full_data_migration`
Step 4 — Record disk state	Broadcom KB 315532
Step 5 — Remove disk group	Broadcom KB 315532 (`esxcli vsan storage remove`)
Step 6 — Power off nested ESXi VM	VMware Workstation standard guest shutdown
Step 7 — Edit outer `.vmx`, add `virtualSSD = "1"`	**VMware KB 1006827 — Configuring SSD virtual disks for VMware Workstation/Fusion**; verified empirically on esxi02 2026-05-09
Step 8 — Power on nested ESXi VM	VMware Workstation standard guest power-on
Step 9 — Verify Is SSD: true	`esxcli storage core device list`
Step 10 — Apply capacityFlash tag	Broadcom TechDocs — Mark Flash Devices as Capacity Using ESXCLI
Step 11 — Re-create disk group	Broadcom KB 315532 (`esxcli vsan storage add`)
Step 12 — Verify both flags	Broadcom TechDocs — verification expects BOTH `IsSSD=1` AND `IsCapacityFlash=1`
Step 13 — Exit MM	Standard ESXi
Step 14 — Wait for resync	Memory `feedback_vsan_resync_first_check`

Procedure status — VERIFIED on esxi02 2026-05-09. Every step in Section 4 below was executed end-to-end on esxi02 and confirmed working. The new disk group UUID after rebuild is `5264837a-ce20-466b-9721-a20167dca634` (created `Sat May 9 23:44:06 2026`); all 4 disks confirmed `In CMMDS: true` and `Is SSD: true`. Sections 5 and 6 (esxi03, esxi04) use the same verified procedure but require live `esxcli vsan storage list` re-verification at the start of each host's run because the pre-flight Section 3.2 may not match current vSAN membership.

Path NOT used (deprecated in this handbook): Earlier revisions documented `esxcli storage nmp satp rule add --satp=VMW_SATP_LOCAL --device <dev> --option "enable_local enable_ssd"` followed by host reboot, citing Broadcom KB 341618. That KB is for **physical** ESXi hosts. On nested ESXi running on VMware Workstation, this path **does not work** — verified on esxi02 2026-05-09 (rule added, host rebooted, `Is SSD: false` persisted). Any pre-existing SATP rules from earlier attempts should be removed with `esxcli storage nmp satp rule remove ...` (cleanup-only — they are inert, not actively harmful, but pollute the rule list).

---

3. Pre-flight Verification

3.1 Cluster state

VERIFIED 2026-05-08 via PowerCLI:

Item	Required	Current
All 4 hosts Connected/PoweredOn	Yes	✓ all 4
HA Enabled	False	✓ False
DRS Enabled	True	✓ True
DRS Automation	Fully Automated	**Currently PartiallyAutomated — change in Step 1**
vSAN Enabled	True	✓ True
Active vSAN resync components	0	**Currently 2 — wait per Step 0**

3.2 Target disks by host (with verified UUIDs)

VERIFIED 2026-05-09 via SSH `esxcli vsan storage list` on each host. Device IDs and Disk Group UUIDs below are exact — no placeholders.

Host	VSAN Disk Group UUID	Cache device	Capacity disks (✓ OK / ❌ misclassified)
esxi01	(already clean — skip)	n/a	All capacity disks SSD=True
esxi02	`520145fe-246d-a3d6-66ec-a4bc0c5efe0c`	`t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____00000000000000000001` (100 GB)	`02...` (500 GB) ✓ | `03...` (400 GB) ❌ | `04...` (100 GB) ❌

Host	VSAN Disk Group UUID	Cache device	Capacity disks (✓ OK / ❌ misclassified)
esxi03	`52923e23-5610-6fab-78d6-6bf2f8f8f91d`	`t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________00000000000000000001` (100 GB)	`02...` (500 GB) ✓ | `04...` (100 GB) ❌
esxi04	`526073df-a49a-886a-87d4-6b45ecb84126`	`t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________00000000000000000001` (100 GB)	`02...` (500 GB) ✓ | `03...` (400 GB) ❌ | `04...` (100 GB) ✓

Device ID prefix shorthand: every disk on every host has the prefix `t10.ATA_____VMware_Virtual_SATA_Hard_Drive__ ________`. The 4 short suffixes are: - `00000000000000000001` = cache - `02000000000000000001` = capacity 500 GB - `03000000000000000001` = capacity 400 GB - `04000000000000000001` = capacity 100 GB

3.3 SSH host key fingerprints (use with `plink -hostkey "..."`)

Host	Fingerprint
esxi02	`SHA256:FCYgFo7Rpesdoe21CNGCGBO9vRm/HEm3y1VYWT6f4C8`
esxi03	`SHA256:vsgF9+pELOr8g+UIGeejw05Dd6XXZ/LHioyzf+79SOs`
esxi04	`SHA256:A50DU+qKpuDZUwKDifEo4iaZ/ap3mghErupaEJFgNQ4`

Password for all three: refer to credential vault (memory `reference_homelab_esxi_creds`). Do not commit or share inline.

3.4 vSAN capacity headroom

VERIFIED 2026-05-08: vSAN datastore `vcenter-cl01-ds-vsan01` has **1.87 TB free of 3.6 TB total (52% free)**. Each host's vSAN data is approximately 600 GB. With FTT=1 RAID1 and one host in MM at a time, the remaining 3 hosts have ample room to absorb the evacuated objects.

3.5 Outer-VM `.vmx` state — verified 2026-05-09

The outer-VM `.vmx` for each nested ESXi guest has SATA disks at slots `sata0:0`, `sata0:2`, `sata0:3`, `sata0:4`. The flag `sataX:Y.virtualSSD = "1"` tells VMware Workstation/Fusion to present that virtual disk as SSD to the guest. **All three target hosts (esxi02, esxi03, esxi04) have the same defect: `sata0:0` and `sata0:2` carry the flag; `sata0:3` and `sata0:4` do not.**

Host	sata0:0 (cache)	sata0:2 (capacity 500GB)	sata0:3 (local.vmdk)	sata0:4 (lab.addon.vmdk)
esxi02	✅ `virtualSSD=1`	✅ `virtualSSD=1`	❌ missing	❌ missing
esxi03	✅ `virtualSSD=1`	✅ `virtualSSD=1`	❌ missing	❌ missing
esxi04	✅ `virtualSSD=1`	✅ `virtualSSD=1`	❌ missing	❌ missing

`.vmx` **paths to edit** (Workstation Library → right-click VM → Open VM directory):

Host	Workstation library entry	`.vmx` filename
esxi02	"Clone of esxi02.lab.local"	`Clone of esxi02.lab.local.vmx`
esxi03	"Clone of esxi03.lab.local"	`Clone of esxi03.lab.local.vmx`
esxi04	"Clone of esxi04.lab.local"	`Clone of esxi04.lab.local.vmx`

Lines to add (paste alongside existing `sata0:0.virtualSSD = "1"` and `sata0:2.virtualSSD = "1"` block):

```
sata0:3.virtualSSD = "1"
sata0:4.virtualSSD = "1"
```

> **Reconcile with §3.2 before each host:** Section 3.2 lists "misclassified" capacity disks based on a 2026-05-09 PowerCLI snapshot. The `.vmx` analysis above shows BOTH `sata0:3` and `sata0:4` are layer-1 broken on every host. Possible explanations: (a) one of those slots is not currently a vSAN capacity disk on that host, (b) the §3.2 snapshot was incomplete. **Before starting Section 5 or Section 6, re-run `esxcli vsan storage list` on the target host and confirm which device IDs are actually members of the vSAN disk group**, then apply the `.vmx` flag to all `sata0:X` slots that map to those device IDs. Edit only the slots whose corresponding disks are vSAN capacity members — adding the flag to a non-vSAN slot is harmless, but the verification table below assumes vSAN-only slots.

3.6 Notes on the `.vmx` files

- All three VMs are clones (`displayName = "Clone of esxi0X.lab.local"`). Edit the `.vmx` for the *running* VM, not the original template.
- esxi03's `.vmx` references the ESXi installer ISO at `E:\VCF-Depot\...`. The depot moved to `I:\VCF-Depot\` on 2026-04-30 (memory `reference_vcf_depot_server`). This is a CDROM image attached at boot; runtime is unaffected. Update opportunistically when the host is powered off for the `.vmx` edit.
- esxi02's `.vmx` already references the depot at `I:\VCF-Depot\...` (correct).
- esxi04's `.vmx` already references the depot at `I:\VCF-Depot\...` (correct).

---

4. Procedure for esxi02 (first host)

esxi02 is the first host because (verified 2026-05-09): it has only **vcf-ops** running on it (a single user VM that tolerates vMotion fine) AND it has 2 misclassified disks — getting the harder host out of the way while the cluster has full headroom.

Step 0 — Confirm resync is zero

Run from: any system that can SSH to inner ESXi. Use the ESXi root password from your credential vault (memory `reference_homelab_esxi_creds`).

```
ssh root@esxi02.lab.local
esxcli vsan debug object health summary get
```

Expected: every category reports `0`.

If non-zero, wait. Re-run every 5-10 minutes. Do NOT proceed to Step 1 until fully zero. Memory `feedback_vsan_resync_first_check`.

Step 1 — Set DRS to Fully Automated (one-time)

Run from: vCenter UI.

1. Inventory → Cluster `vcenter-cl01` → **Configure** → Services → **vSphere DRS** → click **Edit**

2. Automation Level: change from `PartiallyAutomated` to **Fully Automated**
3. Migration Threshold: leave at 3 (default)
4. Click **OK**

Verify: Cluster Summary shows `DRS Automation: Fully Automated`.

Step 2 — SKIP (vCenter already on esxi01)

DRS rebalanced earlier and vcenter is on a non-target host. **No vCenter pre-migrate needed for esxi02.** (Note: vCenter has since moved to esxi03 — that pre-migrate is needed in Section 5, not here.)

Step 3 — Enter Maintenance Mode on esxi02

Run from: vCenter UI.

1. Inventory → right-click `esxi02.lab.local` → Maintenance Mode → **Enter Maintenance Mode**
2. In the dialog: - **vSAN data migration:** select **"Full data migration"** - Check **"Move powered-off and suspended virtual machines to other hosts in the cluster"**
3. Click **OK**

What happens automatically: DRS vMotions vcf-ops + vCLS off esxi02. vSAN evacuates esxi02's data to the other 3 hosts. Wall time: 10-30 minutes.

Verify MM complete:

- Host icon shows the wrench/MM symbol
- PowerCLI: `(Get-VMHost esxi02).ExtensionData.Runtime.InMaintenanceMode` returns `True`

Verify resync is zero before Step 4:

```
 ssh root@esxi02.lab.local
esxcli vsan debug object health summary get
# All categories must be 0
```

Step 4 — SSH and verify disk inventory

```
 ssh root@esxi02.lab.local
esxcli vsan storage list
```

Verified pre-flight values (re-confirm before destructive actions): - VSAN Disk Group UUID: `520145fe-246d-a3d6-66ec-a4bc0c5efe0c` - Cache device: `t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____0000000000000000001` (Is SSD: true) - Capacity 500 GB (OK): `t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____0200000000000000001` - **Capacity 400 GB (MISCLASSIFIED):** `t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____0300000000000000001` - **Capacity 100 GB (MISCLASSIFIED):** `t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____0400000000000000001`

If any of these don't match, STOP and re-verify with PowerCLI.

Step 5 — Remove the disk group on esxi02

```
esxcli vsan storage remove -u 520145fe-246d-a3d6-66ec-a4bc0c5efe0c
```

Verify:

```
esxcli vsan storage list
# Should show NO entries on this host
```

> Per Broadcom KB 315532: "*Always double check the disk group UUID.*" Type the UUID `520145fe-246d-a3d6-66ec-a4bc0c5efe0c` exactly. This UUID belongs to esxi02. Different hosts have different UUIDs.

Step 6 — Power off the nested esxi02 VM gracefully

Run from: VMware Workstation on the host workstation (Precision 7920).

1. Open VMware Workstation.
2. In the Library pane, right-click `Clone of esxi02.lab.local` → **Power** → **Shut Down Guest**. - This sends an ACPI shutdown so ESXi initiates a clean kernel halt. - If "Shut Down Guest" is greyed out, use **Power Off** as fallback (host is in MM with vSAN already evacuated, so a hard power-off is acceptable here).

Verify: the VM tile shows `Powered off`. From the workstation:

```
Test-NetConnection -ComputerName 192.168.1.75 -Port 22 -InformationLevel Quiet
# Expected: False
```

> **Why power off?** Steps 7 below edits the outer `.vmx` file. VMware Workstation reads `.vmx` only at VM power-on; live edits are ignored or silently overwritten on next save. The VM must be off for the edit to stick.

Step 7 — Edit the outer `.vmx`, add `virtualSSD = "1"`

Run from: the host workstation file system.

1. **Locate the running VM's `.vmx`.** In VMware Workstation: right-click `Clone of esxi02.lab.local` → **Open VM directory**. The active config file is `Clone of esxi02.lab.local.vmx`.
2. **Open `.vmx` in a text editor** (Notepad++, VS Code, or `notepad.exe`). Plain UTF-8, no BOM.
3. **Find the existing `virtualSSD` block.** It should look like: `sata0:0.virtualSSD = "1" sata0:2.virtualSSD = "1"`
4. **Add two lines below it** so the block becomes: `sata0:0.virtualSSD = "1" sata0:2.virtualSSD = "1" sata0:3.virtualSSD = "1" sata0:4.virtualSSD = "1"`
5. **Save the file.**

> **Slot mapping is not interchangeable.** The `sata0:3` and `sata0:4` slots are tied to the actual misclassified disks via the `sataX:Y.fileName` lines: - `sata0:3.fileName = "esxi02-local.vmdk"` → vSAN device `t10...030000 00000000000001` - `sata0:4.fileName = "esxi02.lab.addon.vmdk"` → vSAN device `t10...04000000000000000001`
>
> If you ever rebuild the VM with disks attached to different sata slots, re-derive the mapping from the `fileName` ↔ vSAN-device-ID correspondence. **Verified for esxi02 on 2026-05-09.**

Step 8 — Power on the nested esxi02 VM

Run from: VMware Workstation.

1. Right-click `Clone of esxi02.lab.local` → **Power** → **Power On**.
2. Wait for ESXi to fully boot (DCUI banner shows the IP `192.168.1.75`).

Verify host is reachable again from workstation:

```
Test-NetConnection -ComputerName 192.168.1.75 -Port 22 -InformationLevel Quiet
# Expected eventually: True
```

Then re-SSH:

```
ssh root@esxi02.lab.local
# password from credential vault (memory reference_homelab_esxi_creds)
```

The host stays in Maintenance Mode through the power cycle — that's by design (MM is persisted in vCenter, not a local boot flag). After it comes back online, it will still be in MM in vCenter.

Step 9 — Verify Is SSD: true on both disks

```
esxcli storage core device list -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____________03000000000000000001 | grep "Is SSD"
esxcli storage core device list -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____________04000000000000000001 | grep "Is SSD"
```

Both must show: `Is SSD: true`

If either shows `Is SSD: false`: the `.vmx` edit did not take effect for that slot. Do NOT proceed to Step 10. Triage in this order:

1. **Confirm the `.vmx` was actually saved** — re-open and look for both `sata0:3.virtualSSD = "1"` and `sata0:4.virtualSSD = "1"` lines.
2. **Confirm you edited the `.vmx` of the running VM**, not an old/template `.vmx` in a different directory. If Workstation is showing `Clone of esxi0X.lab.local`, the file is `Clone of esxi0X.lab.local.vmx`, NOT `esxi0X.lab.local.vmx`.
3. **Power off, save, power on again.** A live `.vmx` edit while powered on does not stick — it must be off → edit → on.
4. **If still false after a clean cycle:** the disk's `sataX:Y.fileName` mapping may not match the vSAN device ID you're checking. Run `esxcli vsan storage list` to print the mapping again and verify which slot holds which device.

If still failing after all four checks, STOP and see §8 Rollback.

Step 10 — Apply capacityFlash tag to both disks

Run from: SSH session on esxi02. Run BOTH commands:

```
esxcli vsan storage tag add -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____________03000000000000000001 -
t capacityFlash
```

```
esxcli vsan storage tag add -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001 -t
capacityFlash
```

Verify both flags on both disks:

```
for dev in \
  t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001 \
  t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001; do
  echo "--- $dev:"
  vdq -q -d $dev | grep -E "IsSSD|IsCapacityFlash"
done
```

Expected for each device:

```
 "IsSSD": "1"
"IsCapacityFlash": "1"
```

Step 11 — Re-create the disk group with cache + 3 capacity disks

```
esxcli vsan storage add \
  -s t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________00000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________02000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001
```

(Cache disk with `-s`, each capacity disk with its own `-d`. esxi02 has 3 capacity disks total — the already-OK 500 GB plus the 2 freshly-tagged.)

Verify:

```
esxcli vsan storage list
```

All 4 disks (cache + 3 capacity) should show `In CMMDS: true` and `Is SSD: true`.

Step 12 — Verify both flags (vdq + vCenter)

Sub-check A — local vdq verification of all 4 disks:

```
for dev in \
  t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________00000000000000000001 \
  t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________02000000000000000001 \
  t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001 \
  t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001; do
  echo "--- $dev:"
  vdq -q -d $dev | grep -E "IsSSD|IsCapacityFlash"
done
```

Expected: every device shows `"IsSSD": "1"`. The 3 capacity devices (`02...`, `03...`, `04...`) additionally show `"IsCapacityFlash": "1"`. The cache device (`00...`) does NOT have IsCapacityFlash (correct — it's the cache tier).

Sub-check B — vCenter side via PowerCLI on the workstation:

```
Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -Password '<vcenter-administrator-
password>'
$h = Get-VMHost esxi02.lab.local
$sys = Get-View -Id $h.ExtensionData.ConfigManager.VsanSystem
```

```
$sys.Config.StorageInfo.DiskMapping | ForEach-Object {
  Write-Host "Cache:    SSD=$($_.Ssd.Ssd)   $($_.Ssd.CanonicalName)"
  foreach ($cap in $_.NonSsd) {
    Write-Host "Capacity: SSD=$($cap.Ssd)   $($cap.CanonicalName)"
  }
}
Disconnect-VIServer -Confirm:$false -Force
```

Expected — every line ends `SSD=True`:

```
 Cache:    SSD=True   t10...00000000000000000001
Capacity: SSD=True   t10...02000000000000000001
Capacity: SSD=True   t10...03000000000000000001
Capacity: SSD=True   t10...04000000000000000001
```

If any line shows `SSD=False`: STOP. Don't exit MM. See §8 Rollback.

Step 13 — Exit Maintenance Mode on esxi02

Run from: vCenter UI.

1. Right-click `esxi02.lab.local` → Maintenance Mode → **Exit Maintenance Mode**

Or via SSH on esxi02:

```
esxcli system maintenanceMode set --enable false
```

Verify: host icon loses the wrench symbol.

Step 14 — Wait for vSAN resync to zero

```
esxcli vsan debug object health summary get
```

Expected eventually: all categories `0`. Wall time: 10-60 min as vSAN re-mirrors data back to esxi02.

> **Critical:** do NOT start esxi03's MM (Section 5) until esxi02's resync is fully zero. Memory `feedback_vsan_resync_first_check`.

---

5. Procedure for esxi03 (second host)

After esxi02 succeeds and resync is zero, do esxi03. **Critical placement note:** as of 2026-05-09, vCenter VM and Fleet VM both live on esxi03. vCenter must be pre-migrated BEFORE entering MM.

Verified values for esxi03: - VSAN Disk Group UUID: `52923e23-5610-6fab-78d6-6bf2f8f8f91d` - Misclassified disk (1 only): `t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____04000000000000000001` (100 GB capacity)

Step 1 (esxi03) — DRS already Fully Automated from Section 4 — skip

Step 2 (esxi03) — Pre-migrate vCenter from esxi03 → esxi01 (NOT skippable for this host)

Run from: vCenter UI.

1. Inventory → find VM named `vcenter` (currently on esxi03)
2. Right-click `vcenter` → **Migrate**
3. Migration Type: **Change compute resource only**
4. Select destination host: **esxi01.lab.local** (esxi01 has nsx-manager 48 GB only — easily fits vcenter's 18 GB)
5. Click **Next** → review summary → **Finish**
6. Wait ~30 seconds; vCenter UI may stutter briefly mid-vMotion and reconnect

Verify: vcenter VM now shows on esxi01.

(Fleet VM does NOT need pre-migration — DRS will move it during MM. Fleet tolerates vMotion fine.)

Step 3 (esxi03) — Enter Maintenance Mode

vCenter UI → right-click `esxi03.lab.local` → Maintenance Mode → Enter Maintenance Mode → **"Full data migration"** → check Move powered-off VMs → OK.

DRS will auto-vMotion fleet to another host. Wait for MM complete + resync zero before continuing.

Steps 4-14 (esxi03) — Use these specific values

> **Pre-procedure verification.** Before starting, SSH to esxi03 and run `esxcli vsan storage list` to confirm which devices are currently vSAN members. Section 3.2 of this handbook says only `04...` is misclassified, but Section 3.5 shows the `.vmx` is missing `virtualSSD = "1"` for BOTH `sata0:3` and `sata0:4`. The discrepancy means either (a) `sata0:3` (esxi03-local.vmdk) is not currently in the vSAN disk group, or (b) the §3.2 record is incomplete. **Source of truth = the live `esxcli vsan storage list` output.** Adjust the `esxcli vsan storage add` command in Step 11 to match the actual capacity disks reported.

```
ssh root@esxi03.lab.local

# Step 4 — verify disk inventory (CRITICAL — adjust subsequent steps to match):
esxcli vsan storage list

# Step 5 — remove disk group:
esxcli vsan storage remove -u 52923e23-5610-6fab-78d6-6bf2f8f8f91d
```

Step 6 — power off `Clone of esxi03.lab.local` from VMware Workstation (Power → Shut Down Guest).

Step 7 — edit `Clone of esxi03.lab.local.vmx`. Add the missing `virtualSSD` lines so the block reads:

```
sata0:0.virtualSSD = "1"
sata0:2.virtualSSD = "1"
sata0:3.virtualSSD = "1"
sata0:4.virtualSSD = "1"
```

(Add `sata0:3` and `sata0:4` lines — both are missing per §3.5. If only `sata0:4` is currently a vSAN member, the `sata0:3` flag is still harmless and pre-tags it for any future expansion.) **Optional opportunistic fix:** while you're in the `.vmx`, update `sata0:1.fileName` from `E:\VCF-Depot\...` to `I:\VCF-Depot\...` per memory `reference_vcf_depot_server` (depot moved 2026-04-30). Save the file.

Step 8 — power on `Clone of esxi03.lab.local` from VMware Workstation. Wait for SSH on `192.168.1.76`. Re-SSH.

```
 # Step 9 — verify Is SSD: true on the affected slot(s):
esxcli storage core device list -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001 | grep "Is SSD"

# Step 10 — capacityFlash tag (only on devices that are vSAN capacity members):
esxcli vsan storage tag add -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001 -t
capacityFlash

# Step 11 — re-create disk group. ADJUST THE -d LIST to match Step 4 output.
# If esxi03 has 2 capacity disks (per §3.2):
esxcli vsan storage add \
  -s t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________00000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________02000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001
# If Step 4 also showed sata0:3 (esxi03-local) as a vSAN member, ADD:
#   -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001
```

Step 12 — `vdq -q -d <each device> | grep -E "IsSSD|IsCapacityFlash"` should show `IsSSD=1` on all and `IsCapacityFlash=1` on every capacity device. Optional PowerCLI cross-check from workstation (same form as §4 Step 12 sub-check B).

Step 13 — Exit MM via vCenter UI.

Step 14 — Wait for `esxcli vsan debug object health summary get` to show every category at `0` (especially `data-move`) before starting esxi04. Memory `feedback_vsan_resync_first_check`.

---

6. Procedure for esxi04 (third host)

After esxi03 succeeds and resync is zero, do esxi04. esxi04 currently runs sddc-manager (16 GB), the broken VCFA appliance `vcfa-mgmt-wsv57` (96 GB), and a vCLS agent.

Verified values for esxi04: - VSAN Disk Group UUID: `526073df-a49a-886a-87d4-6b45ecb84126` - Misclassified disk (1 only): `t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001` (400 GB capacity)

Step 0' (esxi04) — Power off the broken VCFA appliance BEFORE MM

The broken `vcfa-mgmt-wsv57` is allocated 96 GB. No other host has 96 GB free. DRS will struggle to vMotion it. **Power it off first** — we're going to delete it in Phase 2 anyway.

Run from: vCenter UI.

1. Inventory → right-click `vcfa-mgmt-wsv57` → Power → **Power Off** (force, not Shut Down Guest — guest tools won't respond cleanly given the apiserver crashloop)
2. Confirm

Verify: VM shows PoweredOff in inventory.

Step 3 (esxi04) — Enter Maintenance Mode

vCenter UI → right-click `esxi04.lab.local` → Maintenance Mode → Enter Maintenance Mode → **"Full data migration"** → check Move powered-off VMs → OK.

DRS will auto-vMotion sddc-manager + vCLS to another host. The powered-off vcfa-mgmt-wsv57 will move with the storage migration. Wait for MM complete + resync zero.

Steps 4-14 (esxi04) — Use these specific values

Pre-procedure verification. Before starting, SSH to esxi04 and run `esxcli vsan storage list` to confirm which devices are currently vSAN members. Section 3.2 says only `03...` is misclassified, but Section 3.5 shows the `.vmx` is missing `virtualSSD = "1"` for BOTH `sata0:3` and `sata0:4`. The §3.2 entry also shows `sata0:4` as `04 ✓`, which is internally inconsistent with §3.5 — verify live before destructive action.

```
ssh root@esxi04.lab.local

# Step 4 — verify disk inventory (CRITICAL):
esxcli vsan storage list

# Step 5 — remove disk group:
esxcli vsan storage remove -u 526073df-a49a-886a-87d4-6b45ecb84126
```

Step 6 — power off `Clone of esxi04.lab.local` from VMware Workstation.

Step 7 — edit `Clone of esxi04.lab.local.vmx`. Add the missing `virtualSSD` lines so the block reads:

```
sata0:0.virtualSSD = "1"
sata0:2.virtualSSD = "1"
sata0:3.virtualSSD = "1"
sata0:4.virtualSSD = "1"
```

Save the file.

Step 8 — power on `Clone of esxi04.lab.local` from VMware Workstation. Wait for SSH on `192.168.1.82`. Re-SSH.

```
# Step 9 — verify Is SSD: true on the affected slot(s):
esxcli storage core device list -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001 | grep "Is SSD"
esxcli storage core device list -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001 | grep "Is SSD"

# Step 10 — capacityFlash tag on every vSAN capacity member that lacks it.
# Per §3.2, only 03... was missing the flag. If §3.2 is correct, ONE tag command:
esxcli vsan storage tag add -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001 -t
capacityFlash
# If Step 4 reveals 04... is also a vSAN capacity member without the flag, ALSO tag it.

# Step 11 — re-create disk group with cache + 3 capacity (per §3.2):
esxcli vsan storage add \
  -s t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________00000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________02000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001 \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001
```

Step 12 — `vdq -q -d <each device> | grep -E "IsSSD|IsCapacityFlash"` for all 4 disks. Optional PowerCLI cross-check.

Step 13 — Exit MM via vCenter UI.

Step 14 — Wait for resync zero. Phase 1 complete.

---

7. After all three hosts are done

When esxi02, esxi03, esxi04 all complete Steps 1-10 successfully:

1. **Confirm cluster all-flash:** re-run the PowerCLI verification block from Step 11 against ALL 4 hosts. Every disk on every host should report `SSD=True`.
2. **Verify cluster health:** `esxcli vsan debug object health summary get` returns 0 in every category on every host.
3. **vSAN datastore performance check** (optional but informative): in vCenter, navigate to Cluster → Monitor → vSAN → Performance → **Hosts** → VM consumption tab. Write Latency should drop to under 5 ms; Read Latency under 2 ms; Local Client Cache Hit Rate > 70%. **The 70 ms write latency observed on 2026-05-08 should be gone.**
4. **(Optional) Revert DRS to Partially Automated** before submitting the fresh VCFA deploy. Memory `feedback_no_vmotion_during_vcfa_deploy` documents that vMotion during a VCFA deploy can starve kubelet → kube-vip flap → deploy fails. Switch back via vCenter → Cluster → Configure → vSphere DRS → Edit. After the new deploy COMPLETES, you can flip to Fully Automated again.

Once Phase 1 is verified complete, proceed to **Phase 2 (Delete failed VCFA env)** and **Phase 3 (Fresh VCFA deploy)** per the master runbook `VCFA-Deployment-Fix-Runbook-2026-05-08.md`.

---

8. Rollback Procedure

If any step on any host fails AND you cannot proceed cleanly:

1. **Do NOT panic.** Maintenance Mode with Full Data Migration evacuated all data from the host before disk group removal. The local disk group held no unique data when removed.
2. **If Step 8 (verify Is SSD: true after reboot) shows `Is SSD: false`:** the SATP rule didn't apply. Try the manual reclaim:

```
bash esxcli storage core claiming unclaim --type=device --device=<device-id> esxcli storage core claimrule load esxcli storage core claimrule run esxcli storage core claiming reclaim -d <device-id> esxcli storage core device list -d <device-id> | grep "Is SSD"
```

If still false, you may need a second reboot. If still false after second reboot, investigate the SATP rule itself (`esxcli storage nmp satp rule list -s VMW_SATP_LOCAL`) — the device ID may have a typo.

3. **If Step 10 (re-create disk group) fails:** SSH back to the host and check `esxcli vsan storage list`. If the disk group is partial or absent, remove any remnants with `esxcli vsan storage remove -u <UUID>` and try Step 10 again.

4. **To revert all changes on a host (return to original misclassified state):**

```bash # Remove the disk group: esxcli vsan storage remove -u

# Remove the capacityFlash tag(s): esxcli vsan storage tag remove -d -t capacityFlash

# Remove the SATP rule(s): esxcli storage nmp satp rule remove --satp=VMW_SATP_LOCAL --device= --option="enable_local enable_ssd"

# Reboot host to clear the kernel SSD detection reboot

# After reboot, re-create the disk group WITHOUT the tag: esxcli vsan storage add -s -d -d ```

5. **Exit Maintenance Mode** — vSAN resyncs the original-state data back. The cluster returns to "broken but functional" — same state as before this handbook.
```

Worst case: abort with a host having no disk group. The remaining 3 hosts keep vSAN running. You can re-add the disk group from vCenter UI later (Cluster → Configure → vSAN → Disk Management → Claim disks).

---

9. Monitoring Pattern — vSAN Resync Watcher

This section documents the watcher pattern used during this procedure to detect when `Resyncing components: 0` so the operator can safely enter Maintenance Mode on the next host. The same pattern can be adapted to watch other conditions (apiserver healthz, kube-vip binding, etcd performance, etc.).

9.1 When to use

Use this watcher pattern when: - A long-running async operation needs to be monitored without polling manually - The operator needs a clear "go-ahead" signal at the moment a condition is met - The condition can be expressed as a probe that returns one summary line of state

For this handbook specifically: **between hosts in Phase 1**, after exiting MM on one host, the cluster runs vSAN resync that takes 10-60+ minutes. The watcher fires when resync hits zero so the next host's MM can begin without delay.

9.2 The probe — `esxcli vsan debug object health summary get`

Verified 2026-05-09: PowerCLI's `Get-VsanResyncingComponent` was tried first but **does NOT report components in `Decommissioning` intent state** — it only returns components in active `Rebuild` status. Components moving via `data-move` (the post-DRS-rebalance cleanup state) silently slip past it. The watcher fired prematurely with PowerCLI; switched to `esxcli` for accurate detection.

Reliable probe: SSH to any inner ESXi host and run:

```
esxcli vsan debug object health summary get
```

Output structure (verified):

```
 Health Status                              Number Of Objects
----------------------------------------  -----------------
remoteAccessible                                          0
inaccessible                                              0
reduced-availability-with-no-rebuild                      0
... (other categories)
data-move                                                13   <-- THE KEY METRIC
nonavailability-related-reconfig                          0
... (more categories)
healthy                                                 149
```

The `data-move` row is the single source of truth for resync activity. All other unhealthy categories should also be 0, but `data-move` is the one that the rebalance/decommission flow surfaces in.

Why this metric and not the others?

Metric	What it counts	Useful for
`reduced-availability-with-active-rebuild`	Components actively being rebuilt due to FTT degradation	Rebuild-from-failure scenarios

Metric	What it counts	Useful for
`data-move`	Components being moved (rebalance, decommission, MM evacuation)	**Post-DRS rebalance, MM evacuation aftermath**
`healthy`	Total compliant components	Sanity check that cluster is otherwise OK
PowerCLI `Get-VsanResyncingComponent`	Subset of components in active rebuild status	NOT enough — misses `data-move`

9.3 The bash watcher wrapper

The Monitor tool runs this command. Each `echo` line becomes a notification. SSH'es to esxi03 every 5 minutes and parses the `data-move` count.

Pre-requisite: the operator's host key for esxi03 must be known. Use this fingerprint format with `plink -hostkey "..."`:

```
SHA256:vsgF9+pELOr8g+UIGeejw05Dd6XXZ/LHioyzf+79SOs
```

(Substitute the fingerprint for whichever host you're polling. Each ESXi has a unique fingerprint; capture once per host.)

The watcher script:

```
 export MSYS_NO_PATHCONV=1
HK="SHA256:vsgF9+pELOr8g+UIGeejw05Dd6XXZ/LHioyzf+79SOs"
PW='<esxi-root-password-from-credential-vault>'
echo "[$(date +%H:%M:%S)] vSAN resync watcher armed -- using esxcli data-move on esxi03"
PREV_DM=""
PREV_TS=""
ITER=0
while true; do
  ITER=$((ITER+1))
  OUT=$(plink -ssh -pw "$PW" -batch -hostkey "$HK" root@esxi03.lab.local \
        "esxcli vsan debug object health summary get" 2>/dev/null \
        | grep -E "^data-move|^healthy")
  TS=$(date +%H:%M:%S)
  DM=$(echo "$OUT" | grep "^data-move" | awk '{print $NF}')
  HEALTHY=$(echo "$OUT" | grep "^healthy" | awk '{print $NF}')

  RATE=""
  if [ -n "$PREV_DM" ] && [ -n "$DM" ]; then
    NOW=$(date +%s)
    DELTA=$((PREV_DM - DM))
    DELTA_SEC=$((NOW - PREV_TS))
    if [ "$DELTA_SEC" -gt 0 ] && [ "$DELTA" -gt 0 ]; then
      OBJ_PER_HR=$(awk -v d="$DELTA" -v s="$DELTA_SEC" 'BEGIN { printf "%.1f", d/s*3600 }')
      ETA_HR=$(awk -v r="$DM" -v ph="$OBJ_PER_HR" 'BEGIN { if (ph > 0) printf "%.1f", r/ph; else print "?"
}')
      RATE=" rate=${OBJ_PER_HR}obj/hr eta=${ETA_HR}hr"
    fi
  fi
  PREV_DM=$DM
  PREV_TS=$(date +%s)

  echo "[$TS] iter=$ITER data-move=${DM} healthy=${HEALTHY}${RATE}"

  if [ "$DM" = "0" ] && [ -n "$DM" ]; then
    echo "[$TS] *** RESYNC COMPLETE *** data-move=0 -- safe to enter MM on the next host"
    break
  fi
```

```
  sleep 300
done
```

Key design points:

Element	Why
`sleep 300` (5 min)	Resync state changes every few minutes; faster polling wastes SSH overhead
`plink -batch -hostkey "$HK"`	Non-interactive SSH with verified host key — no MITM risk, no prompts
Parse `data-move` row only	The single source of truth for active rebalance/decommission
Rate computation via `awk`	Bash can't do float math; awk handles it
`DM = 0` AND `-n "$DM"` triggers break	Defensive — empty output (probe failure) doesn't accidentally fire
Text inside `echo` lines	Each line is a notification; pick actionable wording

9.4 Output format and interpretation

Each line is timestamped. Format:

```
[HH:MM:SS] iter=N data-move=K healthy=H rate=RR.Robj/hr eta=EE.Ehr
```

Field meanings: - `iter` — iteration count, increments every poll - `data-move` — count of vSAN objects in active rebalance/decommission (target: 0) - `healthy` — total compliant objects (target: stable or growing as data-move decreases) - `rate` — observed objects/hour completion rate (only shown if positive delta) - `eta` — projection: `data-move / rate` (only shown if rate is positive)

Interpretation:

- `data-move` **dropping,** `healthy` **rising** — resync progressing as expected. Their sum should be roughly constant.
- **Rate trending down across iterations** — vSAN slowing under load. Possible disk pressure on a host or unrelated activity.
- `data-move` **stable,** `healthy` **stable** — resync is paused or stalled. Investigate. Could indicate a degraded disk or a stuck rebalance.
- **Empty output** — SSH or esxcli failed. Watcher recovers automatically next iteration; if persistent, the host may be offline.
- **Final line** `*** RESYNC COMPLETE ***` — `data-move = 0` reached. Your go-ahead to enter MM on the next host.

9.4.1 Cross-checking with vCenter UI

Always cross-reference the `data-move` count against the vCenter UI:

- vCenter → Cluster → **Monitor** → vSAN → **Resyncing Objects**
- "Total resyncing objects" should equal the `data-move` count from esxcli
- "Bytes left to resync" gives byte-level granularity (esxcli only gives object count)
- "Total resyncing ETA" gives vSAN's own estimate

If esxcli says `data-move=0` but vCenter UI shows objects pending, refresh both — likely a UI cache lag of a few seconds.

9.5 Adapting the watcher for other conditions

The same pattern works for any condition that can be probed. Replace the PowerCLI probe with the new check:

Condition	Probe approach	Fire-on
kube-apiserver `/healthz` returns 200	`curl -sk https://192.168.1.91:6443/healthz`	HTTP code = 200
kube-vip VIP bound on eth0	`kubectl get pod kube-vip-* -n kube-system -o yaml | grep "phase: Running"` (via Invoke-VMScript)	phase = Running
etcd apply latency < 100ms	`kubectl logs etcd-* -n kube-system | grep "apply request took too long" | wc -l`	count = 0 over last minute
Fleet env state change	curl Fleet's `/lcm/request/api/v2/requests/<id>`	state = COMPLETED or FAILED
VCFA bundle CR Ready	`kubectl get bundle vra -n prelude -o jsonpath='{.status.phase}'`	phase = Ready

Pattern boilerplate (substitute the probe and the fire condition):

```
 echo "[$(date +%H:%M:%S)] <condition-description> watcher armed"
ITER=0
while true; do
  ITER=$((ITER+1))
  RESULT=$(<your-probe-here> 2>/dev/null)
  TS=$(date +%H:%M:%S)
  echo "[$TS] iter=$ITER result=$RESULT"
  if <your-fire-condition>; then
    echo "[$TS] *** <CONDITION-MET> ***"
    break
  fi
  sleep <interval-seconds>
done
```

Recommended polling intervals:

Type of probe	Interval
Local file or process check	1-5 seconds
ESXi host via SSH	30-60 seconds
vCenter API via PowerCLI	60-300 seconds
Fleet/LCM API	60-90 seconds
Kubernetes API in workload cluster	30-60 seconds

Don't poll faster than the rate of meaningful change in the underlying signal. Resync state changes every few minutes — polling every 5 sec just wastes API calls.

9.6 Stopping the watcher

If you need to abort the watcher manually before it fires:

- The watcher tool used here (`Monitor`) has a `TaskStop` command — invoke with the task id printed when the monitor was armed
- Or wait for the natural break — `COUNT = 0` exits the loop cleanly
- The watcher itself doesn't modify any state; stopping it has zero side effects

10. Cross-references

Reference	Purpose
Broadcom **KB 315532** — How to manually remove and recreate a vSAN disk group	Source of the disk group remove/re-create procedure (Steps 5, 11)
VMware **KB 1006827** — Configuring SSD virtual disks for VMware Workstation/Fusion	Source of the `sataX:Y.virtualSSD = "1"` flag (Step 7 — the canonical fix path for nested ESXi)
Broadcom **KB 341618** — Enabling the SSD option on SSD-based disks not detected as SSD	**NOT used in this handbook.** Documented for *physical* hosts only; verified ineffective on nested ESXi 2026-05-09. Listed for awareness in case of future cross-references.
Broadcom **TechDocs — Mark Flash Devices as Capacity Using ESXCLI**	Source of the `capacityFlash` tag verification expecting BOTH `IsSSD=1` AND `IsCapacityFlash=1`
Broadcom **KB 327008** — Replace failed cache or capacity disk in vSAN OSA	Reference for capacity disk replacement semantics
Broadcom **KB 326857** — vSAN host fails to enter MM with Full data migration	Pre-flight headroom requirement (§3.3)
Internal memory `feedback_vsan_mm_full_data_migration`	Records the EA→journal-abort incident from 2026-05-01; FDM is the documented safe option
Internal memory `feedback_vsan_resync_first_check`	Hard prerequisite to wait for `health summary get` zero
Internal memory `reference_homelab_vsan_misclassified_capacity`	The verified per-host disk inventory from 2026-05-08
Internal memory `reference_homelab_esxi_creds`	SSH credentials for esxi01-04 — do not duplicate inline
Master runbook `VCFA-Deployment-Fix-Runbook-2026-05-08.md`	Phases 1-3 of the broader deploy fix; this handbook is Phase 1 in detail
Source RCA `VCF-Automation-Deployment-Troubleshooting-Report.md`	Section 11.7.13.3 documents the misclassification finding

11. Document Information

Field	Value
Document Title	vSAN OSA Disk Group Rebuild — Step-by-Step Handbook
Prepared By	Virtual Control LLC
Date	May 8, 2026

Field	Value
Document Version	**4.0** — Major revision: corrected primary path from SATP-rule+reboot (which does not work on nested ESXi) to outer-VM `.vmx` `virtualSSD` edit. Verified end-to-end on esxi02 2026-05-09.
Classification	Internal — Lab Environment
vSAN Mode	OSA (Original Storage Architecture), not ESA
Source Broadcom/VMware KBs	315532 (disk group rebuild), VMware KB 1006827 (`virtualSSD` flag for nested ESXi), TechDocs (capacityFlash verification). KB 341618 is referenced but **NOT used** — it is for physical hosts.
Procedure Type	**Single canonical path — no options, no decision trees.** Per host: 15 steps (0-14), wall time 1-2 hours per host (no reboot needed).
Confidence Level	**High — verified end-to-end on esxi02 2026-05-09.** New disk group UUID after rebuild: `5264837a-ce20-466b-9721-a20167dca634` . All 4 disks (cache + 3 capacity) reported `In CMMDS: true` and `Is SSD: true` after the procedure.
Companion Documents	`VCFA-Deployment-Fix-Runbook-2026-05-08.md` (Phases 1-3 overview), `VCF-Automation-Deployment-Troubleshooting-Report.md` (RCA §11.7.13.3)
Revision History	v1.0 (initial) — Tag-only path, marked Medium confidence pending canary on esxi03. v2.0 — Replaced with canonical Broadcom path including SATP rule + host reboot per KB 341618. v3.0 (2026-05-09) — Reordered host sequence: esxi02 first, esxi03 second (with vcenter pre-migrate), esxi04 third (with broken VCFA power-off). v3.1 — Passwords removed; vault references. v3.2 — Added Section 9 (Monitoring Pattern). v3.3 — Watcher probe corrected to esxcli `data-move` (PowerCLI `Get-VsanResyncingComponent` misses Decommissioning intent). **v4.0 (2026-05-09 evening) — MAJOR CORRECTION**: first execution on esxi02 proved the SATP rule + reboot path **does not work** on nested ESXi (rule applied, host rebooted, `Is SSD: false` persisted). Compared `esxi01.vmx` (working) vs `Clone of esxi02.lab.local.vmx` (broken) and identified the missing `sata0:3.virtualSSD = "1"` and `sata0:4.virtualSSD = "1"` lines. Replaced Step 6 (SATP rule add) and Step 7 (reboot) with new Step 6 (graceful VM power-off), Step 7 (`.vmx` edit), Step 8 (VM power-on). All subsequent steps renumbered. Section 1.1 narrative rewritten. Section 1.2 ("Why this single path") rewritten to explain layer-1 vs VMkernel SATP. Section 2 step-source mapping updated. New Section 3.5 added with the verified `.vmx` state of all three target hosts (esxi02, esxi03, esxi04 — all three have the same `sata0:3` + `sata0:4` defect). Sections 5 (esxi03) and 6 (esxi04) rewritten to use the `.vmx` path with explicit pre-procedure verification step (`esxcli vsan storage list` to reconcile §3.2 vs §3.5). Cross-references updated: VMware KB 1006827 added as primary source; KB 341618 demoted to "not used here, physical hosts only."

This handbook was derived from the verified state of the homelab on 2026-05-08 and corrected end-to-end on esxi02 on 2026-05-09. The primary path is the VMware Workstation `virtualSSD` flag in the outer `.vmx` (per VMware KB 1006827). Disk-group remove/re-create commands are taken verbatim from Broadcom KB 315532. The previously-recommended SATP rule + reboot path (KB 341618) was empirically disproven for nested ESXi on Workstation and is documented only for awareness.